Revision Sprints – Arrays, Strings, 2 Pointers, Maps.

LC 189 – Rotate Array

Solution 1 DONE → using extra arrays

BUT I want to do in-place.

[1, 2, 3, 4, 5] k = 2.

itr 1 [5, 1, 2, 3, 4]

itr 2 [4, 5, 1, 2, 3]

BRUTE.

temp 1 = nums[i]
nums[i] = temp 2
temp 2 = temp 1.

temp 1 = ~~1~~ ~~2~~ ~~3~~ ~~4~~ 5

temp 2 = ~~5~~ ~~1~~ ~~2~~ ~~3~~ ~~4~~ 5

i = ~~1~~ ~~2~~ ~~3~~ 4

last = 5

1 2 ~~3~~ ~~4~~ ~~5~~
2 3 4

[5, ~~1~~, ~~2~~, ~~3~~, ~~4~~]
1 2 3 4

TLE.

We need to bring it in below n^2.

⇒ [1, 2, 3, 4, 5] k = 2.

⇒ [4, 5, 1, 2, 3]

SINGLE ITERATION?

① [5, 4, 3, 2, 1] → Reversed array

↓

② [4, 5, 3, 2, 1] → Reversed first k elements

↓

③ [4, 5, 1, 2, 3] → Reversed last n-k elements.

LC152 – Max Product Subarray

BRUTE FORCE

[2, 3, -2, 4]

↑ ↑ ↑ ↑

↑↑ ↑ ↑

≠ 6 ✓

-12 -3 -2 4.

-48 -6 -8

-24

BETTER → Rolling Prod.

[2 , 3 , -2 , 4]

Basically
i → locked
j → multi into product.

① product *= nums[j]

② compare prod with global max.

They have not given "Cant use divide"
↓
So can we try and use it

[2, 3, -2, 4].

→ Dont think. we can use it → bcoz

even if we find value of subarray by dividing total_prod by prod_pref * prod_suffix

we would still need to → INCREASE length of subarray under

consideration

which basically takes us to $O(n^2)$ again.

LC 53 – Maximum Sum Subarray

$\downarrow$

$\boxed{\text{Kadane's Algorithm}}$

BRUTE → find sum of every sub-array.

BETTER → find sum using rolling-sum.

OPTIMAL → Kadane's Algo.

① If the curr_sum < 0 then restart the sum to zero.

→ do this in a linear traversal.

LC 283 – Move Zeroes

[0, 0, 0, 1, 2, 3, 0]

1, 0, 0 0, 2, 3, 0

l r

swap

1, 2, 0, 0, 0, 3, 0

l r

swap

1, 2, 3, 0, 0, 0, 0

l r

1, 2, 3, 4.

① Initialize l at first zero | r at l+1.

② while r < n && l < n.

r = l+1.

if r != 0.

- swap (nums[l], nums[r]).

// find next zero.

```
while (nums[i] != 0 && i < n)
    i++.
}.
```

non_zero_pointer.

zero_pointer.

loop to find first zero. → set zero_p → index

non_zero_p = zero_p + 1;

LC 169 — Majority Element - I

→ Brute: Pick every element i → count for it

→ Better: Freq-Map

→ Optimal → Moore's Voting Algo.

Element = _

Count = _

1. Pick any nums[i] & set it as element
2. If you find same number → count ++
3. Diffr number → count --
4. If count reaches 0

PICK The next number as <element>

& set <count = 1>

The <element> after the traversal is the POTENTIAL most voted element.

MOST IMP: we have to VERIFY if this element is REALLY the element which occurs > n/2 times

do it using a single traversal O(n).

LC 229 — Majority Element II

→ Replicate moore's voting algo for 2 numbers

↓

BE CAUTIOUS ABOUT

2 elements should not be same or in that case - count inc. & dec. would be unequal.

LC 215 — K^{th} largest element in array.

MAN → QA.

current price = future value
*
discount factor.

discount_curve

time	d_factor

LC - 31 — Next Permutation.

— Brute.

L12 - Recursion

1. Generate all sets using recursion
2. Search the given perm.
3. Return next.

For n digits the diff^n numbers created are n!

∴ for 3 digits given we have 6 permutations.

Better

STL in C++ provides next_permutation ().

Optimal

Intuition -

① We need to keep "LONGER PREFIX MATCH" such that we find a "BREAK POINT"

which is basically a place where

arr[i] < arr[i+1] → if this is NOT the

case then we CANNOT rearrange numbers to find next biggest

Hence, we <u>NEED</u> a larger number in the <u>AREA</u> which is NOT a prefix.

Eg: $[2, 1 \| 5, 4, 3, 0, 0]$

i, i+1

BREAK POINT

(i) we traverse in reverse direction so that we can maximize prefix.

(ii). Now we have 3 parts in array.

$[\underbrace{\overset{①}{2}}_{\text{prefix}}, \underbrace{\overset{②}{1}}_{\text{update this to NEXT smallest}}, \underbrace{5, \overset{③}{4}, 3, 0, 0}_{\text{options for update}}]$

(2) Now find next smallest from the

options. so that PART ② gets finalized

↓

③ for PART ③ of array → rearrange ALL remaining numbers in SORTED order

↓

because that will give the smallest number.

Algo:

1. Find break point by for loop.
2. Start a reverse traversal to find a no. GREATER than part ② number.

   ↓

   SWAP it.

3. Reverse the part ③

Notes : Intuition behind step 2 & 3 is that the part (3) will always be in order → which is basically descending (we're seeing from left to right).

LC - 3 — Longest Substring Without repeating characters.

→ Sliding Window

→ update L when r is found in map

↓

This makes sure that we don't have s[r] in the substring under consideration.

LC – XX Merge Sort

→ recursive splitting using "low", "mid", "end".

→ merge by sorting using 2 pointers. each in one of the arrays.